

{
Trabalho 1 - (TL)
Prof. Poli

Odd-Even Sort

Relatório Manuscrito

Discentes: Caio Lene, Gabriel Phillipy,
João Pedro Sabino, Matheus Daurado
e Renan Moraes }

1 - Introdução e lógica detalhada do Odd-Even Sort

1.1 - O problema de ordenação

A ordenação de dados é um dos principais problemas na área da Ciência da Computação, sendo crucial para estruturar e organizar informações. Quando realizada em uma sequência específica, seja crescente ou decrescente, ela permite operações mais eficientes, como busca binária, busca por intervalos, agrupamento, filtragem e remoção de dados duplicados. Por essa razão, diversos algoritmos de ordenação foram desenvolvidos ao longo dos anos, sendo alguns deles estudados em sala, como o Bubble Sort, Insertion Sort, Selection Sort, Merge Sort e Quick Sort e, neste trabalho, estudaremos o Odd-Even Sort.

1.2 - Conceito e lógica do algoritmo

O Odd-Even Sort é um algoritmo de ordenação baseado em comparações entre pares de elementos adjacentes. Possui semelhança com o Bubble Sort, mas seu diferencial está na forma como organiza essas comparações, alternando entre duas fases distintas a cada passagem: par e ímpar.

O algoritmo começa declarando e inicializando uma variável de controle, do tipo inteira, chamada "flag" com o valor 1 (verdadeiro) para forçar a entrada no loop externo (while), pois o loop só é executado enquanto o valor da variável "flag" for diferente de 0 (falso), e sem essa inicialização o algoritmo iria encerrar imediatamente sem ordenar nenhum elemento. Dentro do while, no início de cada passagem, "flag" é redefinida como 0 (falso), assumindo que o vetor está ordenado.

Em seguida, o algoritmo executa a fase par por meio de um loop for que percorre os índices pares do vetor, começando em 0 e avançando de 2 em 2, até o tamanho do vetor - 2, garantindo que o último par comparado não ultrapasse o limite do vetor. Em cada iteração, o elemento do índice atual é comparado com seu vizinho à direita, nos pares (0,1), (2,3), (4,5) e assim em diante. Logo após, executa a fase ímpar por meio de um segundo loop for que percorre os índices ímpares, começando em 1 e avançando de 2 em 2, e também até o tamanho do vetor - 2, comparando os pares (1,2), (3,4), (5,6) e assim em diante, também com a mesma lógica da fase par, exceto

pela iteração nos índices ímpares.

Nas duas fases, caso o elemento da esquerda seja maior que o da direita, os dois são trocados de posição, garantindo a ordenação em ordem crescente. Para realizar essa troca sem perder os valores, utiliza-se uma variável auxiliar que armazena temporariamente o valor do elemento da esquerda, para que o elemento da esquerda possa armazenar o valor do elemento da direita e, posteriormente, o elemento da direita irá armazenar o valor que está na variável auxiliar ou seja, o valor do elemento da esquerda. Sempre que uma troca é realizada, a variável "troca" é redefinida como 1 (verdadeiro), indicando que o vetor não está completamente ordenado e irá precisar de uma nova passagem.

No final de cada passagem completa, o loop externo verifica o valor da variável "troca". Quando uma passagem ocorre sem nenhuma troca, a variável "troca" permanece como 0 (falso), pois no início do loop externo foi definida desse modo, e então o loop externo é encerrado e o algoritmo finaliza com o vetor ordenado.

2. Pseudocódigo em nível abstrato

Esta seção apresenta o pseudocódigo do algoritmo em nível abstrato, descrevendo os passos lógicos de forma independente de linguagem de programação. Segue abaixo o pseudocódigo

Algoritmo OddEvenSort (vetor[], tamanho)

troca ← verdadeiro

Enquanto troca = verdadeiro faça

troca ← falso

// Fase par: compara pares de índices (0,1), (2,3)...

Para i de 0 até tamanho-2, passo 2 faça

Se vetor[i] > vetor[i+1] então

aux ← vetor[i]

vetor[i] ← vetor[i+1]

vetor[i+1] ← aux

troca ← verdadeiro

Fim se

Fim para

// Fase ímpar: compara pares de índices (1,2), (3,4)...

Para j de 1 até tamanho-2, passo 2 faça

Se vetor[j] > vetor[j+1] então

aux ← vetor[j]

vetor[j] ← vetor[j+1]

vetor[j+1] ← aux

troca ← verdadeiro

Fim se

Fim Para

Fim Enquanto

Fim Algoritmo

3- Análise Comparativa

Esta seção compara o Odd-Even Sort com Bubble Sort, Insertion Sort, selection sort, Quick sort e merge sort.

3.1- Desempenho

Em execução padrão (sequencial), o Odd-Even sort possui complexidade $O(n^2)$ no pior e no caso médio, equivalente ao Bubble Sort, Insertion Sort e selection Sort, e $O(n)$ no melhor caso. É, portanto, inferior ao Quick Sort e ao Merge Sort, ambos com complexidade média $O(n \log n)$. O algoritmo se destaca no contexto paralelo, reduzindo a complexidade a $O(n)$ em um processo com n processadores.

3.2- Facilidade de implementação

O Odd-Even Sort é um dos algoritmos mais simples de implementar. Em simplicidade é o próximo ao Bubble Sort, Insertion Sort e Selection Sort, e a diferença ao Quick Sort e Merge Sort, que exigem recursividade, particionamento, no caso do Merge Sort, manipulação de memória auxiliar para a intercalação.

3.3- Estabilidade

O Odd-Even Sort é estável, pois trata os elementos apenas entre os elementos adjacentemente próximos de ordem, preservando a ordem relativa de elementos iguais. Com-

partilha uma propriedade com Bubble Sort, Insertion Sort e Merge Sort, mas não com Quick Sort e Selection Sort.

3.4 - Caracter de uma

A característica natural do Odd-Even Sort é o processamento paralelo: GPUs, sistemas SIMD, redes de ordenação e hardware dedicado, mas quando seu problema é explorado com implementações diretas. Em execução sequencial, o algoritmo raramente é o melhor escolha, já que o Insertion Sort tende a superá-lo em entradas pequenas ou quase ordenadas, o Quick Sort é preferível em entradas grandes pelo desempenho médio e ordenação in-place, e o Merge Sort é adequado quando estabilidade e desempenho garantido é o principal requisito, como em ordenação externa.

4 - Análise de complexidade do algoritmo

Esta seção apresenta a análise de complexidade do Odd-Even Sort considerando os casos melhor, médio e pior, bem como a complexidade de espaço, expressas em notação Big-O.

4.1 - Complexidade de Espaço

O Odd-Even Sort é um algoritmo in-place: utiliza apenas uma variável auxiliar (aux) para realizar trocas e uma variável de controle (troca), independentemente do tamanho da entrada. Portanto, sua complexidade de espaço é $O(1)$. (Ambas as variáveis de tamanho constante independentes de n).

4.2 - Complexidade de Tempo - Melhor caso: $O(n)$

O melhor caso ocorre quando o vetor já está completamente ordenado. Nessa situação, o laço externo executa exatamente uma vez.

- Se for par percorre os índices de 0 até $\text{tam}-2$ com passo 2, realizando $\lfloor n/2 \rfloor$ comparações, sem nenhuma troca.
- Se for ímpar percorre os índices de 1 até $\text{tam}-2$ com passo 2, realizando $\lfloor (n-1)/2 \rfloor$ comparações, sem nenhuma troca.

O total de comparações numa única iteração é aproximadamente $n-2$ comparações. E como a variável [troca] permanece [FALSO] ao final, o laço encerra imediatamente. O número de operações cresce linearmente com n , resultando em complexidade $O(n)$.

4.3 - Complexidade de Tempo - Pior caso: $O(n^2)$

O pior caso ocorre quando o vetor está em ordem completamente decrescente. Nessa situação, cada elemento precisa ser deslocado até sua posição correta, exigindo o número máximo de iterações do laço externo.

O laço externo executa no máximo n iterações. Em cada iteração completa (fora par + fora ímpar), são realizadas aproximadamente $n-2$ comparações, conforme demonstrado na seção anterior.

Q total de comparações no pior caso é, portanto:

$$n \times (n-1) = n^2 - 2n$$

Descartando os termos de menor ordem, a complexidade de tempo no pior caso é $O(n^2)$.

4.4 - Complexidade de Tempo - caso médio: $O(n^2)$

No caso médio, o vetor está parcialmente desordenado. Embora o número de iteração do laço externo seja menor do que no pior caso, ele ainda cresce proporcionalmente a n à medida que o tamanho da entrada aumenta.

Considerando uma distribuição aleatória dos elementos, o número esperado de iteração é da ordem de $n/2$, e cada iteração ainda realiza aproximadamente $n-2$ comparações. O total esperado de comparações é:

$$(n/2) \times (n-1) = (n^2 - 2n)/2$$

Que em notação Big-O resulta em $O(n^2)$.

4.5 - Complexidade de Tempo - Execução Paralela: $O(n)$

Em um modelo de execução paralela com n processadores disponíveis, o comportamento do algoritmo muda significativamente. Em cada fase (par ou ímpar), os pares de índices comparados são completamente independentes entre si — os pares $(0,1), (2,3), (4,5), \dots$ não compartilham elementos, portanto todas as comparações e trocas de uma mesma fase podem ser executadas simultaneamente.

Com n processadores, cada fase para a ter custo $O(1)$ (tempo constante). Como o laço externo ainda executa $O(n)$ iterações, o custo total é:

$$n \text{ iterações} \times O(1) \text{ por fase} = O(n)$$

Esta é a principal motivação para o uso do Odd-Even Sort em contextos de computação paralela, como GPUs e redes de ordenação, onde esse paralelismo natural pode ser diretamente explorado.

4.6 - Resumo das complexidades

Caso	Complexidade de Tempo	Complexidade de Espaço
melhor	$O(n)$	$O(1)$
médio	$O(n^2)$	$O(1)$
Pior	$O(n^2)$	$O(1)$
Paralelo	$O(n)$	$O(1)$

5. Exemplos ilustrativos

5.1 Vetor de entrada - o vetor de entrada a ser ordenado:

7	2	5	1	8	4
---	---	---	---	---	---

 tam = 6

[0] [1] [2] [3] [4] [5]

Iteração 1

Fase ímpar: Compara os pares de posições (1,2), (3,4) e (5,6) ou seja, índices ímpares com seus vizinhos à direita

7	2	5	1	8	4
---	---	---	---	---	---

 - Compara [1] com [2], como 2 é menor que 5, não troca

[0] [1] [2] [3] [4] [5]

7	2	5	1	8	4
---	---	---	---	---	---

 - Compara [3] e [4], como 1 é menor que 8, não troca

[0] [1] [2] [3] [4] [5]

Fase par: Compara os pares das posições (0,1), (2,3), (4,5) ou seja, índices pares com seus vizinhos à direita

7	2	5	1	8	4
---	---	---	---	---	---

 →

2	7	5	1	8	4
---	---	---	---	---	---

 - Compara [0] com [1], como 7 é maior que 2, eles trocam

[0] [1] [2] [3] [4] [5] [0] [1] [2] [3] [4] [5]

2	7	5	1	8	4
---	---	---	---	---	---

 →

2	7	1	5	8	4
---	---	---	---	---	---

 - Compara [2] com [3], como 5 é maior que 1, eles trocam

[0] [1] [2] [3] [4] [5] 0 1 2 3 4 5

2	7	1	5	8	4
---	---	---	---	---	---

 →

2	7	1	5	4	8
---	---	---	---	---	---

 - Compara [4] com [5], como 8 é maior que 4, eles trocam

[0] [1] [2] [3] [4] [5] [0] [1] [2] [3] [4] [5]

Iteração 2

Fase ímpar

2	7	1	5	4	8
---	---	---	---	---	---

 →

2	1	7	5	4	8
---	---	---	---	---	---

 - Compara [1] com [2], como 7 é maior que 1, eles trocam

[0] [1] [2] [3] [4] [5] [0] [1] [2] [3] [4] [5]

2	1	7	5	4	8
---	---	---	---	---	---

 →

2	1	7	4	5	8
---	---	---	---	---	---

 - Compara [3] com [4] como 5 é maior que 4, eles trocam

[0] [1] [2] [3] [4] [5] [0] [1] [2] [3] [4] [5]

fase par

2	1	7	4	5	8
---	---	---	---	---	---

 $\rightarrow$

1	2	7	4	5	8
---	---	---	---	---	---

 - compara [0] com [1], como 2 é maior que 1, eles trocam

1	2	7	4	5	8
---	---	---	---	---	---

 $\rightarrow$

1	2	4	7	5	8
---	---	---	---	---	---

 - compara [2] com [3], como 7 é maior que 4, eles trocam

1	2	4	7	5	8
---	---	---	---	---	---

 - compara [4] com [5], como 5 é menor que 8, não trocam

Esta lógica se repete até que em uma iteração, não se tenha mais trocas tanto na fase par, quanto na fase ímpar, significando que o vetor está ordenado

___/___/___

S T Q Q S S D

6-Descrição da colaboração do grupo

- Caio Lene: realizou a comparação do Odd-Even Sort com os demais algoritmos de ordenação estudados em sala e colaborou na revisão da apresentação de slides.
- Gabriel Phillip: ficou responsável por construir o pseudocódigo para a apresentação de slides, comentou no código implementado em C e participou da construção dos slides.
- João Pedro Sabino: realizou o desenvolvimento do Odd-Even Sort em C e ficou responsável pelo pseudocódigo em nível abstrato, código de implementação em C e exemplos ilustrativos demonstrando o comportamento do algoritmo.
- Matheus Dourado: responsável pela análise de complexidade e análise de paralelismo do algoritmo e participou da revisão dos slides e do pseudocódigo.
- Renan Moraes: responsável pela revisão do código desenvolvido em C e pela análise detalhada da lógica do algoritmo, além de contribuir na construção da apresentação de slides.

7. Código de implementação em linguagem C

O código é composto pela função `OddEvenSort`, que recebe o vetor e o seu tamanho como parâmetros. A ordenação ocorre por meio de duas varreduras alternadas a cada iteração do laço principal: uma sobre os índices pares, realizando trocas quando necessário. O processo se repete até que nenhuma troca seja efetuada em uma varredura completa. Segue abaixo o código

```
#include <stdio.h>
void OddEvenSort(int A[], int tam);
void OddEvenSort(int A[], int tam){
    int troca = 1;
    while(troca){
        troca = 0;
        for(int i = 0; i <= tam - 2; i += 2){
            if(A[i] > A[i+1]){
                int aux = A[i];
                A[i] = A[i+1];
                A[i+1] = aux;
                troca = 1;
            }
        }
        for(int j = 1; j <= tam - 2; j += 2){
            if(A[j] > A[j+1]){
                int aux = A[j];
                A[j] = A[j+1];
                A[j+1] = aux;
                troca = 1;
            }
        }
    }
}
```